

Backend Take-Home: Event Trigger Engine

Context

Listenery runs AI-led user research interviews. Customers connect Listenery to their product so that when something happens to one of their users such as when they churn, activate, try a new feature an event fires, and depending on how that customer has configured things, an interview link goes out to that user. This engine is the piece between the incoming event and the outgoing interview.

What to build

1. Ingest

POST /events accepting {event_name, user_id, email, phone, properties, timestamp}, authenticated per client. Accept quickly and return, processing should not happen on the request path.

2. Match

Each client configures rules mapping an event name to a specific interview (such as a churn type interview created by a specific user and tied to a churn event from their product).

3. Schedule

A rule may specify a delay such as send immediately, after 24 hours, after 30 days or custom durations. Previously scheduled dispatches must survive any server or process restarts.

4. Sample

A rule may specify a percentage. Sampling must be deterministic per user: the same user is always on the same side of the cut for a given rule.

5. Dispatch

When a dispatch is due, call a sender function. Do not integrate a real email or SMS provider. Write a stub that logs the payload (client, user, contact, interview ID, timestamp) and returns success.

6. Deduplicate

The same user should not receive the same interview twice inside a configurable window.

Deliverables

- Working code with a README that gets it running from scratch
- Tests for the sampling and deduplication logic
- A short note on your approach, what you skipped, and what you would do with more time

Ground rules

- Python/Django or FastAPI, your choice. PostgreSQL and Redis available.
- Use any AI coding tools you like.

What happens next: A follow-up session will cover a walkthrough of your code, along with some changes to the requirements to talk through together.